AI Containment Assurance Levels (AI-CAL)

A Framework for Assessing AI Agent Containment Strength

Conceptual Reference Architecture and Evidence Requirements

Author: Patrick Shannon
CISSP, CISA, CRISC, GISP, GPEN, GCLD, CDPSE, GPPA, Security+
Principal Security Architect Consultant
LinkedIn: https://www.linkedin.com/in/patrick-shannon-cissp-crisc-cisa-gisp-gpen-gcld-cdpse-b7726b2/
Credly: https://www.credly.com/users/patrick-shannon.d59e52e2/badges/credly

Version 2.0 | November 11, 2026

Status: Proposed AI-CAL framework and conceptual reference architecture. The October 13 Camel/mTLS experiment demonstrated a limited message sequence using simulated components. It did not qualify a deployment level, integrated containment, or production capacity. The signed v1.0 remains a separate release.

Core Design Intent

Compromise may be expected. Unconstrained pivoting is not.

A local boundary breach is an event. Containment fails when prohibited reach, authority, or effects extend beyond the boundary under assessment. A shared dependency can defeat several rings at once.

Patrick Shannon
Digitally signed by Patrick Shannon
Date: 2026.11.11 15:18:05 -05'00'

© 2026 Patrick Shannon. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

Document Control

Document type	Conceptual reference architecture / proposed assurance model
Primary reference profile	AI-CAL 4.5 (enhanced high-assurance hybrid containment)
Audience	Security architects, AI/ML platform architects, infrastructure architects, red teams, security engineering, risk/governance
Status	Version 2.0; proposed framework and conceptual reference architecture. Integrated validation pending.
Purpose	Define the containment levels, the AI-CAL 4.5 reference profile, and the evidence needed to assess a deployment. The companion assessment method and interface contracts cover detailed qualification and implementation.

Contents

1. Executive Summary
2. Status, Scope, and Non-Claims
3. Threat Model and Design Premise
4. Cross-Domain Containment as a Risk Response
5. Terminology and Requirements Language
6. Proposed Containment Assurance Levels (AI-CAL 0–6)
7. Control Envelope and Qualified Capacity
8. Core Architectural Invariants
9. AI-CAL 4.5 Reference Architecture
10. Forward Control Conduit: mTLS, F5, and Owl Data Diode
11. Trusted Request Classification and Package Request Branch
12. Package Broker, JFrog, Managed File Transfer(MFT), and Return-Side Diode Flow
13. Network, F5, Firewall, Routing, and Management-Plane Controls
14. Logging, TAPs, IDS/NDR, and Auditability
15. Hardening Baselines and Boundary Components
16. Scaling, Sharding, and Failure Domains
17. Differences Between AI-CAL 4, 4.5, 5, and 6
18. Design Evolution and Decision Record
19. Open Design Items and Validation Plan
20. Relationship to Emerging Industry Guidance
Model Transaction and Response Sequence
Conclusion on Deployment Assurance
Appendix A. Example Configuration Language
Appendix B. AI-CAL Assessment Checklist
Appendix C. Selected Public References

1. Executive Summary

AI-CAL assesses containment for autonomous AI agent workloads, including workloads that may behave maliciously. It asks what a compromised agent, harness, tool, delegate, or supporting service can reach and do after an inner control fails. Set the required assurance from the use case and potential consequences. Support any claimed level with evidence for the declared boundary, failure conditions, and operating load.

The primary reference profile is AI-CAL 4.5. Hostile execution and trusted mediation use separate physical host pools. The agent submits bounded requests through FW1 to a fixed HTTPS service that requires mutual TLS (mTLS). F5 authenticates the client and derives the agent binding. Camel #2 verifies the signed registration and durably admits the request before F5 releases it through the forward data diode. Camel #1 verifies the reconstructed request and dispatches authorized model or package work. It signs the model result and sends it through a separate one-way return service. Camel #2 verifies the result before F5 releases it to the bound agent. Jenkins retrieves approved packages through JFrog and hands them to managed file transfer (MFT) for one-way delivery to the agent's scoped SFTP landing. The package response is limited to a transfer-submitted notice or silence.

AI-CAL evaluates what the deployed controls enforce: which paths and authorities are reachable, how failures propagate, and whether defenders can detect and stop the effects. The example uses physical boundaries and Camel mediation. Equivalent components must demonstrate the same functions and satisfy the same evidence requirements.

The agent reaches model and package services only through brokers. Each required protocol break terminates the local exchange and starts a separate authorized exchange. The cell holds only its scoped mTLS credential and, where enabled, a separate SFTP pickup key. Provider, repository, signing, and administrative credentials stay outside the cell.

Key distinction: A component name or diagram does not establish assurance. A deployment must demonstrate which boundary controls enforce the declared restriction, whether failures share a bypass or administrator, and whether the remaining controls can contain, observe and stop the resulting consequence.

- AI-CAL rates containment assurance. Hazard and consequence analysis sets the required profile. The companion assessment method distinguishes required strength, demonstrated strength, evidence, confidence, and resilience.
- Report Qualified Capacity separately from the level. State the workload, duration, concurrency, and degraded modes under which the environment can admit, mediate, observe, and stop the work. A fleet of roughly 700–1,000 agents is a planning target.
- The agent's own A001 label or identity field is not authoritative. The ingress derives a bound principal from the validated per-agent certificate and controlled infrastructure mapping. Downstream services authorize each requested operation separately, and the return is checked against durable original state.
- Large-scale hostile-agent tests should not exceed the demonstrated control envelope of the environment. Concurrency itself is treated as an **attack multiplier.**
- Runtime admission, mediation, state and cutoff are part of AI-CAL's proposed enforcement pattern. Prompt-injection defense, sensitive-data handling, delegated tool authorization, model safety and wider governance require complementary controls. An injection may still initiate a containment challenge.

2. Status, Scope, and Non-Claims

2.1 Conceptual status

This version defines a conceptual reference architecture. The October 13 experiment exercised parts of the F5/Camel message sequence using Camel JVMs and mTLS to an F5 stand-in. The model and diode were simulated.

Integrated validation must cover the physical boundaries, product integrations, recovery, and the containment and capacity claims at the intended workload.

The framework addresses agent execution, harnesses, tools, delegates, message paths, supporting services and the authority those paths can exercise. Model-serving infrastructure, data privacy, prompt-injection defense, supply-chain assurance and organizational governance also require their own controls and evidence.

STAT-01 SHALL: Describe this design as a conceptual reference architecture until an integrated implementation has been validated.

STAT-02 SHALL: Describe AI-CAL 0–6 as a proposed containment assurance model. Its levels are distinct from regulatory classifications, published standards, and third-party certification schemes.

2.2 Scope

- Autonomous or semi-autonomous AI agents executing potentially hostile code or cybersecurity tasks.
- Hostile-agent evaluation environments such as exploit research, benchmark sandboxes, malware-analysis-like tasks, or high-autonomy red-team experiments.
- Enterprise development agents where a lower AI-CAL 1–3 may be sufficient.
- On-premises, private infrastructure, and hybrid architectures where the organization has direct authority to configure, enforce, and verify the controls that define the agent containment boundary. This includes network controls across OSI Layers 1-7 and the infrastructure that supports them: hypervisors, switches, routers, firewalls, application delivery, monitoring, and cross-domain transfer systems. The reference architecture emphasizes direct control of physical connectivity and switching at the physical and data link layers (Layers 1-2). Hybrid deployments must identify where organizational control ends and which controls are operated by a provider.

The declared containment boundary must name prohibited destinations, operations, data movement and effects. Include developer input, task assignment, agent completion, output export and human review in the scope analysis even when their detailed contracts are still proposed.

2.3 Non-claims

- Residual risk remains, including the possibility of hypervisor escape.
- Select the required level for the workload and consequences. AI-CAL 4.5, 5, and 6 are intended for cases that justify their additional controls.
- Serial/Moxa and MQTT are historical candidates. Assess any alternate conduit against the intended profile.
- A hardware data diode enforces transfer direction within the assessed configuration. Validate the proxies, management routes, return paths, and receiving-side content controls separately.
- The design does not require the example vendors named in this document. Vendor names illustrate implementable roles.

3. Threat Model and Design Premise

Assume the agent can persist, adapt, and exploit unintended infrastructure behavior. Its guest operating system, test target, local process, or supporting service may be compromised. The design limits what a local compromise can reach or do next.

Information starvation is a bounded feedback policy. Invalid or unauthorized transactions should not return parser coaching, broader routes or additional capability. The agent may still observe a failure, timeout or local state, and local computation can continue after a conduit is cut. Rate, session and cumulative-work limits constrain adaptation and load. Independent controls must stop active and queued effects when required.

Goal-Seeking Compliance and Deterministic Failure Constraints

Before execution, the agent receives operating rules and notice of qualifying violations and possible throttling, suspension or cutoff. This may encourage compliant use of approved conduits, but motivation is not an enforcement control. The boundary must independently validate identity, message class, authorization, state and path.

- Malformed or unauthorized requests are denied under a bounded feedback contract and logged on the defender side. Schema adherence alone neither proves benign intent nor authorizes an action.
- Observed attempts to cross a declared boundary trigger investigation and, under defined policy, a scoped enforcement action. A blocked probe is an anomaly; evidence of prohibited reach or effect is needed before calling it a containment failure.
- Credential revocation can block new authenticated admission. Full cutoff also accounts for open sessions, already-dispatched model or package work, queued jobs, local execution, mailbox pickup and effects that cannot be recalled.
- A containment failure occurs when the declared prohibited reach, authority or effect is realized across the boundary. One exploit or common dependency may defeat multiple nominal layers; the number of steps in the attack chain is not the definition.

Reference Technology Choices

The example uses Camel, Kubernetes, F5, Owl, JFrog, Jenkins, and GoAnywhere MFT. Validate the selected releases, product capabilities, and integrations. Section 18 records Node-RED, Moxa/serial, and MQTT as historical alternatives.

AI-CAL specifies security functions, trust boundaries, containment properties, and evidence requirements. Alternative products can fill the same roles when their enforcement and failure behavior meet those requirements.

- Agent may obtain root or equivalent authority inside its own VM/container.
- Agent may exploit kernels, virtual device drivers, runtime parsers, package services, orchestration components, or other reachable services.
- Agent may probe virtual NIC, storage, memory-management, or hypervisor interfaces if exposed.
- Agent may discover unintended shared state and use it to communicate with other agents.
- Agent may retry and adapt quickly enough that human response alone is not a sufficient boundary.
- Agent may attempt to use permitted services for unintended purposes, including data exfiltration, coordination, or lateral movement.
- Any custom component can fail or be compromised. Failure of one control MUST NOT automatically grant broad authority in the next ring.

Design premise: A control is more valuable when its failure removes capability or reveals an anomaly rather than silently granting a broader capability.

4. Cross-Domain Containment as a Risk Response

Containment is not unique to AI or cybersecurity. Across physical safety and security domains, containment is used when an uncontrolled escape, release, or transition could extend harm beyond an intended boundary. Prison systems, nuclear or radiological environments, and biological containment laboratories manage fundamentally different hazards, yet they share a recognizable risk-management pattern: layered boundaries, controlled transitions, independent safeguards, and increasing restrictions as the consequence of failure rises.

These examples are conceptual analogies, not operational equivalencies. They do not imply that correctional security, nuclear security and safety, biosafety and biosecurity, and AI systems use the same architecture, terminology, engineering controls, or regulatory requirements. The comparison is intentionally limited to the common containment principle: when what is inside can create significant consequences outside the boundary, responsible design uses precautions proportional to the hazard and does not assume that one successful boundary crossing should grant access through the next.

Cross-domain containment principle: Grade restrictions by consequence and control the transition across the declared boundary. Independent barriers can limit propagation after a local breach; a shared administrator, state service, platform or route can defeat several nominal barriers at once.

4.1 Shared Containment Characteristics

The three physical examples are useful because each begins with a different hazard but converges on similar architectural behavior. The subject or hazard is intentionally present inside a controlled environment. The surrounding system is designed so that movement toward or away from the protected center occurs through successive boundaries, each with its own access conditions, monitoring, and response expectations.

- **Layered boundaries.** Perimeter, intermediate, and inner zones create successive barriers. Crossing one layer does not inherently confer the ability, authority, or knowledge needed to cross the next.
- **Controlled transitions.** Movement between layers is mediated by distinct procedures and controls. Access at one boundary is not automatically inherited at the next boundary.
- **Risk changes by layer.** Controls are selected according to what the next boundary protects or contains and the consequence if that boundary is crossed.
- **Independent failure domains.** The architecture assumes individual safeguards can fail and seeks to prevent one failure from collapsing the remaining containment structure.
- **Detection and intervention.** Transitions and boundaries are monitored so abnormal movement, release, or attempted bypass can be detected and interrupted before consequences propagate further.
- **Proactive risk treatment.** Containment is established before the harmful event. The objective is to make consequence-limiting controls part of the environment rather than depend solely on response after escape or release.

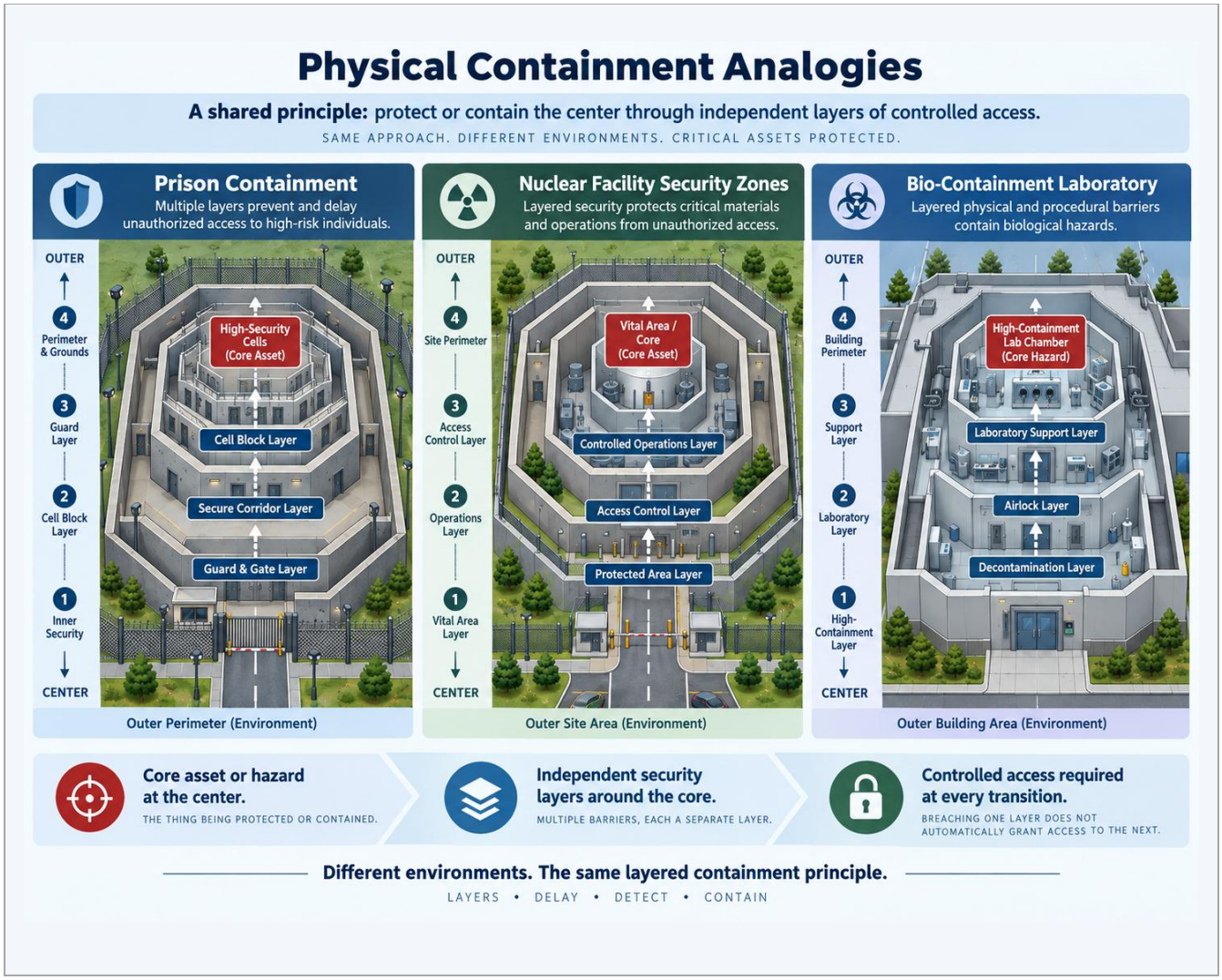


Figure 1. Conceptual comparison of layered containment across prison, nuclear/radiological, biological, and AI risk domains.

4.2 Contained Hazard Characteristics

The comparison becomes more useful when the hazards are evaluated by the behavior of consequences after containment fails rather than by the identity of the hazard itself. The matrix below is deliberately qualitative. It is not a severity ranking and should not be interpreted as a technical model for correctional, nuclear, or biological risk. Its purpose is to expose common containment variables that also matter for autonomous AI systems.

Contained hazard / domain	Primary concern if containment fails	Propagation / reach and speed	Observability / persistence	Containment implication
High-risk individual / correctional	Harm to people or public safety following unauthorized escape or movement.	Primarily physical movement; reach expands through accessible locations and transportation at human-scale speed.	Usually observable as an escape or unauthorized movement; threat persists while the individual remains uncontrolled.	Layered physical access, controlled movement, delay, detection, and repeated authorization between zones.
Nuclear / radiological	Exposure, contamination, theft or misuse of material, or disruption of critical operations.	Consequence can remain local or extend broadly depending on material, event, environment, and pathway; timing is event-dependent.	Hazard may be invisible without instrumentation; contamination or operational consequences can persist well beyond the initiating event.	Independent barriers, graded zones, monitoring, and controls proportional to the consequence of unauthorized access or release.

Contained hazard / domain	Primary concern if containment fails	Propagation / reach and speed	Observability / persistence	Containment implication
Biological	Exposure, infection, contamination, or environmental/public-health consequences.	Reach and speed depend on the agent, route of exposure, transmission characteristics, and environment; effects may emerge over hours, days, or longer.	Release may not be immediately apparent; some biological hazards can persist or amplify after leaving the original boundary.	Multiple physical and procedural barriers, controlled transfer, decontamination, and increasingly restrictive handling as consequence rises.
Autonomous AI capability	Unauthorized digital actions that create cyber, operational, financial, physical, or societal consequences.	Effective reach is determined by connectivity, interfaces, credentials, tools, delegated authority, and downstream automation; digital actions can occur at machine speed.	Boundary crossing may be difficult to observe or attribute immediately; issued actions, copied artifacts, or downstream processes may persist after the original agent is stopped.	Constrain capability, authority, and pathways; mediate transitions; isolate failure domains; independently observe boundaries; prevent one compromise from enabling broad pivoting.

Matrix note: *The examples illustrate shared containment variables only. They do not assert equivalent severity, regulatory treatment, propagation mechanics, or facility design across the domains.*

4.3 Propagation and Consequence as Containment Variables

Containment risk is not exhausted by the question, "Can the boundary be breached?" The consequence profile after a breach determines how urgently the next layers must limit progression. Across domains, several variables influence the required strength of containment:

- **Propagation reach or radius.** How far can consequences extend once the original boundary is crossed?
- **Propagation velocity.** How quickly can the hazard or its effects move beyond the original containment area?
- **Observability.** How quickly and confidently can defenders determine that containment has been crossed or that harmful effects have begun?
- **Persistence.** How long can the hazard or its effects remain after the initiating source has been isolated or removed?
- **Amplification.** Can the initial event reproduce, scale, trigger downstream processes, or otherwise increase its impact without a proportional increase in human effort?
- **Path dependence.** What infrastructure, access, environmental conditions, interfaces, credentials, tools, or intermediaries are required for further propagation?
- **Reversibility.** After an external effect has occurred, can it be reliably recalled, canceled, remediated, or returned to the prior state?

These variables help explain why containment depth is risk-dependent. A low-consequence workload with narrow reach and easy reversibility does not require the same containment assurance as a workload whose failure could propagate rapidly, invisibly, and across multiple downstream systems.

4.4 Implications for AI Containment

AI introduces a digital containment problem with the potential for real-world consequence. Unlike a physical subject, an autonomous digital capability does not need to move physically to expand its effective reach. Its propagation radius can be determined by the interfaces it can address, the credentials and authority it can exercise, the tools it can invoke, the networks it can reach, and the systems or people that will act on its outputs.

The digital nature of the hazard can also compress time. Actions may be generated, repeated, and distributed at machine speed, while the resulting effects may be indirect, delayed, difficult to attribute, or already queued in downstream systems before defenders recognize that a boundary has been crossed. For this reason, containment should not be evaluated only by the strength of the initial sandbox. It should also evaluate what a successful compromise can reach next, how quickly consequences can propagate, how independently the next boundary is enforced, and whether defenders can observe and terminate progression.

The shared design principle is to strengthen the boundary when uncontrolled capability can create greater consequences outside it. Each domain implements that principle in its own way.

Application to AI-CAL: Use hazard and consequence analysis to select the required strength. Behavioral, model, and runtime controls can reduce likelihood. Containment controls limit the consequences of a local breach and give defenders ways to observe and stop further progression.

5. Terminology and Requirements Language

5.1 Core terms

Term	Definition	Use in this design
AI-CAL	AI Containment Assurance Levels. Proposed tiers for the strength of containment enforcement.	AI-CAL 0–6. A defined intermediate profile, such as AI-CAL 4.5, is distinct from a software version or a calculated score.
Ring	An architecture label for a functional process containment position. A ring is not by itself an independent trust, physical or failure domain.	Ring 0 agent/landing, Ring 1 response mediation, Ring 2 trusted request mediation, Ring 3 package assurance, Ring B boundary devices and Ring M management/monitoring.
Zone	A purpose-specific security compartment within or adjacent to a ring.	Trusted ingress, request relay, model gateway, package broker, landing zone, management zone.
Tollgate	An enforcement point for a transition across a trust boundary. Each tollgate permits only approved operations and supplies no automatic authority at the next crossing.	A firewall restricts reach. F5 terminates the agent's mTLS session and releases a separately authorized onward request only after Camel #2 admits it.
Cell	A horizontally isolated agent execution unit and its assigned test resources.	Agent Cell A001, A002, etc.; each cell receives a dedicated workload identity and bounded test resources.
Lane	A deterministic communication path whose identity is established by infrastructure provenance.	mTLS client identity -> F5 trusted-ingress mapping -> Owl one-way conduit -> trusted relay/API route.
Conduit	A narrow, purpose-specific path crossing a boundary.	Typed forward request, controlled model response and separately governed package artifact return; physical return hardware may be shared subject to policy and failure analysis.
Control Envelope	The demonstrated combination of agent capability, concurrency, connectivity, duration, monitoring, and termination capacity that an environment can safely support.	Authorization boundary for a specific experiment.

Term	Definition	Use in this design
Qualified Capacity	Maximum concurrent agent/cell load demonstrated at a claimed AI-CAL and control envelope.	Reported separately from AI-CAL, for example: AI-CAL 4.5 / qualified for 32 concurrent hostile cells.
Trusted Origin Provenance	Trust in authenticated infrastructure evidence and observed path rather than agent-supplied identity or routing claims.	Validated mTLS certificate principal plus observed source/F5 policy/data diode/relay mapping is authoritative; A001 remains a correlation tag.
Physical Channel Provenance (alternate pattern)	Trust in a fixed physical path rather than agent-supplied identity claims when a serial or other dedicated physical conduit is used.	Applies to the retained lower-density serial reference pattern; physical port/channel mapping is authoritative in that implementation.

Termination ends a session or protocol context at its enforcing endpoint. Re-origination starts a separate, authenticated exchange. A harness is the local task and tool execution layer around the model. Enforcement independence depends on separate authority, bypass resistance, and failure paths. Evidence records the configuration, observed behavior, and operating conditions supporting a claim.

5.2 Requirements language

SHALL / MUST = required for a claimed reference profile. **SHOULD** = expected unless an explicit risk decision justifies an alternative. **MAY** = optional implementation choice.

6. Proposed Containment Assurance Levels (AI-CAL 0–6)

Whole-number levels describe major changes in the accepted failure domains. A defined intermediate profile describes a specific architecture between those levels. AI-CAL 4.5 is the intermediate profile used in this reference architecture.

Level	Summary	Typical enforcement	Illustrative use
AI-CAL 0	Unassured / uncontained	Agent runs with user-equivalent desktop authority, credentials, network position, or broadly shared environment.	Not acceptable when containment is required.
AI-CAL 1	Basic isolation	Basic sandbox or constrained environment; limited permissions; material shared dependencies may remain.	Low-risk experiments or basic agent isolation.
AI-CAL 2	Development containment	Dedicated container/sandbox, separate agent identity, limited mounts, constrained network/tool access.	Typical developer-team agent use with bounded business impact.
AI-CAL 3	Strong virtual containment	Dedicated VM/cell, strong micro-zoning, brokered capabilities, centralized logging, no general-purpose egress.	Higher-risk enterprise agents or security tooling.

Level	Summary	Typical enforcement	Illustrative use
AI-CAL 4	High-assurance hybrid containment	Separate guests/zones, purpose-bounded conduits, stateful firewall enforcement, trusted ingress/proxy controls, MFT, TAP/IDS, and directional control hardware where justified. Virtualization may still be a shared failure domain.	Hostile-agent research where operational practicality still matters.
AI-CAL 4.5	Enhanced high-assurance hybrid	Separate physical hostile/trusted hypervisor hosts/pools; narrowly routed mTLS ingress; physical stateful firewall; hardened F5-class ingress; independent hardware one-way diode path(s); trusted relays may remain virtual.	Primary reference profile for scalable high-assurance containment.
AI-CAL 5	Maximum-assurance physical containment	Critical containment functions on independent physical systems/appliances; virtualization is not relied upon as the critical boundary; dedicated hardware directional/protocol-break links used where practical.	Exceptional/high-consequence hostile-agent research.
AI-CAL 6	Fully air-gapped / sovereign containment	No live external model, Internet, enterprise, cloud-control-plane, or remote-management dependency during operation; inference and support services reside inside isolated boundary.	Rare, highly resourced environments where external dependency is unacceptable.

Assessment rule: Claim a level or defined intermediate profile only after meeting its mandatory criteria. The proposed numeric scoring rubric remains subject to validation.

Level assignment is a gate, not an average of present controls. Name the mandatory profile criteria, declared boundary and exceptions; an absent mandatory control prevents that level claim until the criterion is met or the profile is explicitly revised. The companion assessment method proposes measures for strength, resilience and evidence. Its numeric rubric and thresholds are not yet ratified.

The level table is conceptual. A control catalog mapping, vendor feature or successful isolated test does not establish that a deployment satisfies a whole level. AI-CAL 4.5 remains the specific hybrid reference example; AI-CAL 5 adds independent physical systems for critical functions, and AI-CAL 6 removes live external dependencies during operation.

7. Control Envelope and Qualified Capacity

Containment strength and scale are separate. For the declared use case, compare required strength with demonstrated strength, evidence confidence, failure resilience and the measured capacity at which the controls still enforce and observe the boundary.

CAP-01 SHALL: Agent concurrency shall not exceed the demonstrated containment, monitoring, and emergency-termination capacity of the environment.

CAP-02 SHALL: A test authorization shall document the intended agent capability, autonomy, external connectivity, duration, concurrency, and potential consequence, then compare that workload with the environment control envelope.

CAP-03 SHALL: An AI-CAL claim shall report Qualified Capacity separately; for example, "AI-CAL 4.5 / qualified for 32 concurrent hostile cells."

CAP-04 SHOULD: Increase concurrency incrementally and requalify after material changes to the infrastructure, agent capability, model behavior, or external service access.

- Use Case Hazard + Agent Capability + External Capabilities + Concurrency + Potential Consequence
    - Required Containment Assurance
- Demonstrated strength, visibility, cutoff and Qualified Capacity at the intended workload and failure conditions?
    - YES -> authorize only within the measured envelope and approved residual risk
    - NO -> reduce workload or authority, improve controls, and retest before claiming capacity

Can the organization predict, observe, constrain, and terminate the effects of this workload at the intended scale and under the declared failure conditions?

The assessment record should identify physical, virtual, cloud and management dependencies, active requests, pending responses, queue and storage limits, and the tested degraded modes. Ownership of a control or an architectural intention is not evidence that it works under load.

8. Core Architectural Invariants

INV-01 SHALL: No hostile agent cell shall receive general-purpose external network access. Any hostile-side network interface shall be restricted to explicitly approved boundary endpoints, ports, and static/connected routes.

Rationale: The design removes arbitrary DNS, dynamic routing adjacencies, provider/API reachability, and broad east-west access while still permitting a narrow HTTPS/mTLS path to the trusted ingress boundary.

INV-02 SHALL: Each cross-boundary logical conduit shall have bounded direction, identities, message classes, destinations and policy. Model response and package artifact functions remain separately authorized and observed even if qualified physical return hardware is shared.

Rationale: The forward conduit carries only bounded typed control requests. The model-response and artifact-return contracts have different authorities, content and destinations; shared equipment is a common dependency to assess.

INV-03 SHALL: Request classification may occur in one constrained trusted relay, but model-response, package-delivery, repository, provider-credential, and management authorities shall remain separated by role, identity, and zone.

Rationale: A compromised parser/dispatcher must not automatically inherit model-provider credentials, repository authority, artifact-delivery authority, or management-plane capability.

INV-04 SHALL: The agent shall not directly access JFrog/Artifactory, the model-provider API, or management interfaces for MFT, Jenkins, package brokers, or infrastructure.

Rationale: Supporting services are capabilities, not neutral conveniences.

INV-05 SHALL: Agent-supplied cell identifiers, source addresses, return addresses, routes, or provider destinations shall not be authoritative for routing or access decisions.

Rationale: Trusted origin is derived from validated mTLS certificate identity and infrastructure-observed/mapped path provenance; untrusted copies of those fields are replaced at the boundary.

INV-06 SHALL: Invalid forward requests shall not receive parser details or corrective information across the request conduit.

Rationale: The one-way request path should fail silent toward the hostile cell while producing defender-side telemetry, reducing information leakage and preventing parser feedback from becoming a capability.

INV-07 SHALL: Critical management planes shall be isolated from data planes and hostile paths.

Rationale: F5, Owl, firewalls, hypervisors, trusted relays, API gateways, MFT, repository services, and diode administration shall not be reachable from hostile data paths.

INV-08 SHALL: Failure of the trusted ingress, diode adapter, or relay service shall remove capability rather than expose a shell, administrative interface, alternate listener, direct provider route, or broader fallback service.

Rationale: The permitted path must fail toward reduced capability under component outage or malformed work. Availability and safety consequences of denial still require defined response and recovery.

INV-09 SHALL: A hardware one-way path shall not be treated as a reversible session. Opposing flows shall use separate controlled conduits, endpoints, identities, and policies.

Rationale: Independent forward and return paths preserve directionality and reduce common-mode compromise or accidental route-back.

INV-10 SHOULD: Security-critical boundaries should use independent observation such as passive TAPs where practical.

Rationale: Compromise of an endpoint or software sensor should not erase all evidence of boundary traffic.

INV-11 SHOULD: Diversity should be applied selectively across critical failure domains, not indiscriminately.

Rationale: Different OS/firewall implementations may reduce common-mode risk, but excessive heterogeneity creates operational risk.

INV-12 SHALL: The environment shall fail toward reduced agent capability.

Rationale: Loss of F5 ingress, the forward diode, trusted request relay, model response path, or package path removes the associated capability; it shall not create an alternate network route.

INV-13 SHALL: Reject noncompliant requests without exposing parser details, new routes, or additional capability to the agent.

Rationale: A denial or timeout can still be observable. Cutting a model path does not stop local execution or work already queued.

The effectiveness of these invariants depends on the whole path, including administration, failover, return services and state recovery. Shared credentials, a state store or a management route can create a common-mode bypass despite separate ring labels. Developer intake and output release require their own authority and review contracts.

9. AI-CAL 4.5 Reference Architecture

AI-CAL 4.5 is the primary reference profile. Hostile agents and Camel #1 occupy separate physical host pools. Camel #2 runs in its own agent-side mediation pool and controls durable admission and response state. FW1–FW4, a physical F5-class ingress tier, and forward and return Owl-class one-way services enforce the intended crossings. The reference deployment uses Apache Camel on hardened Kubernetes. A measured level depends on the implemented controls and their evidence.

Ring and zone convention: Rings name containment roles; zones, interfaces, identities and physical placement state the actual boundary. Ring B groups boundary functions without granting a flat subnet. Ring M is the separate management and monitoring plane. The numbering below follows the October 2026 architecture decision log.

9.1 Current ring and zone vocabulary for the reference example

Label	Purpose	Examples
Ring 0 Agent and scoped landing	Hostile execution and narrowly scoped local pickup.	Agent cells, assigned targets, read-only SFTP mailbox.
Ring 1 Response mediation	Durable admission, original recipient binding, return state and behavioral enforcement.	Camel #2 in a separate agent-side mediation host pool.

Label	Purpose	Examples
Ring 2 Trusted request mediation	Validate reconstructed requests and authorize the model/package branches.	Camel #1, approved model gateway interactions.
Ring 3 Package assurance	Catalog/policy, acquisition, scanning, manifest and transfer staging.	Jenkins, JFrog/Artifactory, MFT and package assurance services.
Ring B Boundary devices	Enforce narrowly specified crossings and directionality.	FW1–FW4, F5, forward and return diode endpoints, TAPs.
Ring M Management and monitoring	Administration, independent visibility, response and recovery.	Controlled consoles, OOB control, SIEM and sensor management.

FW1 constrains agent and local landing paths; FW2 governs Camel #1 and request-service crossings; FW3 governs package services and relevant trusted-side egress/return interfaces; FW4 protects Camel #2 and the limited F5 registration/delivery exchanges. A repeated firewall symbol in a flow view may denote a different interface of the same device, not another appliance.

10. Forward Control Conduit: mTLS, F5, and Owl Data Diode

An agent initiates a bounded HTTPS transaction to the fixed F5 service through FW1 using mTLS. F5 authenticates the per-agent certificate, terminates that TLS session, and derives the principal from a controlled mapping. F5 submits the signed registration record, including the agent binding, transaction ID, canonical request digest, and expiry, to Camel #2 through FW4. Camel #2 verifies the record and durably admits the pending request. Only an accepted, unexpired internal acknowledgment permits F5 to release the request toward the forward diode and Camel #1. A denial or an unavailable admission service blocks release. Validate this contract on the selected BIG-IP integration.

FWD-01 SHALL: Authenticate each cell at the trusted ingress with its dedicated mTLS client identity. Enforce the approved certificate trust chain, validity and revocation policy, and certificate-to-cell mapping before admission.

FWD-02 SHALL: The F5-class ingress tier shall terminate the hostile-side TLS session and construct a new bounded request representation. Agent-supplied trusted_origin, source_ip, return_ip, routing, provider, or destination claims shall be discarded or replaced with infrastructure-derived values.

FWD-03 SHALL: The model/provider destination, API credential, repository destination, and downstream service route shall not be supplied by the agent.

FWD-04 SHALL: Authoritative origin identity shall be derived from the validated client certificate and mapped infrastructure provenance, not from a CELL_ID field controlled by the agent.

FWD-05 SHALL: The forward request path shall traverse an independent hardware one-way protocol-break boundary before reaching the trusted request relay in the AI-CAL 4.5 reference profile.

FWD-06 SHALL: Use digitally signed application records to preserve authenticated origin and payload integrity across proxy termination, protocol reconstruction, and one-way transfer. The receiving Camel service shall verify the trusted signer, record integrity, and transaction binding before accepting or routing the record.

Camel #2 returns its admission acknowledgment to F5 over an authenticated local service channel. This acknowledgment does not represent a model result or a final response to the agent. Local TLS sessions terminate at the proxies. The forward and return diode services use separate hardware directions.

Camel #1 verifies the reconstructed request before dispatch. It signs the model result and binds it to the original transaction and request digest. Camel #2 verifies the signature and checks the original binding, expiry, and duplicate state. If the authenticated request is still open within its bounded wait, F5 can release one inline result. After a timeout, the agent can authenticate to the Response VIP to retrieve the existing result without reissuing model work. Durable release state governs the race between the two paths.

Illustrative signed request envelope. The interface contract defines the exact encoding, digest input, and signed bytes:

```
{
  "schema_version": "<selected version>",
  "transaction_id": "<infrastructure-generated ID>",
  "request_type": "MODEL_REQUEST | PACKAGE_REQUEST",
  "agent_binding": "<infrastructure-derived principal>",
  "request_digest": "<digest of canonical request bytes>",
  "expires_at": "<bounded expiry>",
  "payload_length": "<bounded size>",
  "payload": "<typed bounded content>",
  "key_id": "<trusted signer key identifier>",
  "signature": "<signature over the defined record>"
}
```

An authorized research workload may carry adversarial test data, debugging output, or package identifiers. Boundary components validate identity, size, framing, message type, and route authority. The content remains untrusted and subject to workload and downstream policy.

11. Trusted Request Classification and Package Request Branch

Camel #1 verifies the signed request binding, payload digest, and allowed message class after the forward one-way transfer. It holds model and package-job correlation. Camel #2 holds the original recipient, delivery, repetition, and strike state. Camel #1 sends MODEL_REQUEST to the approved model gateway and PACKAGE_REQUEST to the catalog policy and Jenkins workflow. Neither role provides general forwarding or inherits the other role's administrative authority.

The reference deployment calls for Apache Camel on hardened, horizontally scaled Kubernetes services, with separate identities, network policies, and durable state. The implementation must select the state store, network plugin, and packaging. Assess any hybrid variant against its actual private routes, authority, telemetry, and failure conditions.

REQ-01 SHALL: The trusted request relay shall accept only explicitly defined request types and shall not expose a general forwarding, proxy, shell, URL-fetch, or arbitrary command capability.

REQ-02 SHALL: Normalization shall occur only after the entire request is proven compliant with the defined grammar. The relay shall not guess intent or repair non-compliant requests.

REQ-03 SHALL: Unknown fields, malformed framing, invalid encodings, oversize values, prohibited request types, or missing mandatory fields shall cause discard/quarantine and defender-side logging without parser detail returned on the forward request path.

REQ-04 SHALL: The canonical package job shall contain only approved catalog coordinates and bound transaction/agent metadata. It shall not copy hostile bytes, arbitrary URLs, commands, repository targets or agent-supplied routing into Jenkins or MFT work.

REQ-05 SHOULD: Artifact and manifest staging should use a completion marker or atomic promotion so a receiver cannot consume a partial object. The exact file and transfer contract remains an implementation decision.

Illustrative PACKAGE_REQUEST payload after trusted-origin fields have been separated from package semantics:

```
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
```

11.1 Bounded response and lifecycle extensions

F5 does structural and ingress checks; Camel #2 performs durable admission and recipient binding; Camel #1 performs trusted branch validation. Authentication proves a peer identity, while each component's authorization decision limits the permitted request, destination and state transition. A source IP, JSON schema or agent-supplied label alone is insufficient authority.

For chunked uploads, enforce limits on each transaction and the entire object. A trusted staging service tracks object and chunk IDs, order, count, digests, expiry, duplicate handling, and cleanup of incomplete objects. A 10 MB transaction and a 50 MB object are illustrative. Verify the gateway's file, context, and streaming limits before setting policy; dispatch only after the broker verifies completeness.

Authorized developer input is bound to an assigned job and agent after intake and assurance review. A read-only mailbox may deliver a completed artifact and manifest, but mailbox text has a defined trust level and cannot silently change the task. The harness needs an explicit pickup checkpoint. Agent completion is a bounded status claim through an already permitted path; independent verification and trusted recipient selection precede notification. Result export uses a separately authorized path and isolated human review, not write access to the input mailbox.

Scoped Boundary Secrets and Transport-Layer Cryptography

The cell holds its per-agent mTLS identity and, if package pickup is enabled, a separate SFTP pickup key. Scope, rotate, and revoke those credentials independently. Keep enterprise, cloud, provider, signing, and administrative credentials outside the cell.

- The return diode adapters use separate local SSH sessions and keys for SFTP. The agent uses a different key to read its own mailbox. An SSH session terminates at its local endpoint and does not traverse the diode.
    - SFTP transfer and pickup shall use SSH public-key authentication only. Disable password and keyboard-interactive authentication, including password prompts through PAM. Transfer accounts shall have no shell, command execution, forwarding, or terminal access. See Section 12 and the OpenSSH reference [30].
- F5 maps the client certificate to a principal. Camel #2 and the downstream policy services separately authorize transactions and actions. Section 11.2 defines integrity and key separation across the protocol breaks.

11.2 Signed Application Messages and Integrity Verification

- **Forward requests:** The authorized ingress signer shall sign the registration binding before Camel #2 admission. Camel #2 shall verify the signature before recording acceptance. After the admission acknowledgment, F5 shall forward the same signed binding with the canonical request payload. Camel #1 shall verify the binding and recompute and match the request digest before dispatch.
- **Returned records:** Camel #1 shall sign the return record. Camel #2 shall verify the signer, signature, and response digest, then match the transaction and original request digest to the pending record and bound agent before authorizing release.
- **Signature coverage:** The defined signed bytes shall bind the schema version, message type, security-sensitive identifiers, relevant digests, and validity fields. Carry the signature and verification metadata in the application contract across protocol reconstruction. If using Camel Crypto, explicitly serialize and recover its signature metadata at the interface [31].
- **Failure handling:** Reject missing or invalid signatures, untrusted signing keys, altered payloads, expired records, and inconsistent bindings before forwarding or release. Record the reason on the defender side. Enforce replay and duplicate checks separately.
- **Key separation:** Keep infrastructure signing private keys outside agent cells and separate from transport and SFTP credentials. Each receiving role shall trust only verification keys authorized for that direction and message class. A valid signature authenticates the record; the receiving tollgate still authorizes the action.
- **Implementation validation:** Specify and test the encoding, algorithms, signing-service placement, key custody, rotation, revocation, and proxy behavior. Test how a compromised or revoked signer affects each downstream authority.

F5 enforces the admission acknowledgment before release. The ingress signature binds the request to its signer and contents. To claim that downstream admission enforcement survives ingress compromise, the receiving boundary must verify evidence of Camel #2 admission that the ingress cannot forge.

12. Package Broker, JFrog, MFT, and Return-Side Diode Flow

The package path separates intent, authorization, acquisition, and delivery. Camel #1 classifies the admitted PACKAGE_REQUEST. The package policy service authorizes exact coordinates from an approved catalog. Jenkins

retrieves the approved artifact through the selected JFrog/Artifactory, Xray, and Curation workflow, verifies the bytes, and hands the artifact, manifest, and job trigger to GoAnywhere MFT. Validate these integrations on the selected product releases.

PKG-01 SHALL: MFT shall move request and artifact objects according to fixed source/destination policy; MFT shall not decide whether a package is authorized.

PKG-02 SHALL: The Package Broker shall decide whether the requested package/version/target is authorized and shall translate the logical request into the real repository operation.

PKG-03 SHALL: The agent shall not provide the JFrog URL, repository name, credentials, shell command, package-manager arguments, proxy settings, or arbitrary download URL.

PKG-04 SHALL: Scope the package broker identity to approved repositories or repository views. Compromise of that identity shall not grant access outside its assigned scope.

PKG-05 SHOULD: The approved Jenkins/package workflow should resolve permitted dependencies, verify exact bytes, generate a manifest with sizes and digests, and stage a complete bounded bundle for MFT. Receiving-side assurance remains separate.

PKG-06 SHALL: Verify artifact bytes against an expected digest from an authenticated manifest or approved catalog. Protect the reference digest from modification by transfer and landing services. Integrity verification does not establish that an artifact is safe to run.

PKG-07 SHALL: Use the hardware data diode to enforce direction. Validate and scan the reconstructed artifact on the receiving side before promotion.

PKG-08 SHALL: Compromise of a landing service, including its local administrator, shall not enable unauthorized delivery or signaling between isolated cells through the pickup path. Enforce that separation outside the landing service's administrative authority. A shared host with authority over several mailboxes is a shared failure domain; per-agent keys and directories on that host do not establish independence.

MFT sends the approved bytes through FW3, the package return diode, and FW1 to the scoped SFTP landing. Camel #2 and F5 carry no package bytes. An authenticated MFT event can support one fixed notice that the transfer was submitted. Submission establishes neither landing nor pickup. The best-effort contract has no SFTP-origin callback and makes no claim that the artifact was installed or executed.

Each local SSH client shall verify the server's host identity. Require public-key authentication on SFTP transfer and pickup endpoints; disallow direct root login and restrict transfer identities to file transfer. Agent pickup is read-only and scoped to the assigned mailbox. This removes the password guessing path; enforce connection and work limits separately for availability [30].

A nonreceipt claim may permit a bounded resend of the same retained, approved artifact. Stable agent and artifact identities carry repetition limits across new request IDs and reconnects. Excess repetition pauses affected transfers and alerts defenders, while ordinary authorized retries and service failures are not automatically policy strikes.

For Python, the hostile cell should install from a local approved wheelhouse rather than reaching PyPI or JFrog directly. A representative pattern is:

```
pip install --no-index --find-links=/packages <approved-package>
```

Git/GitHub is not required in this runtime package-request path unless the organization chooses to store pipeline definitions, catalogs, or infrastructure-as-code there for administrative change control.

13. Network, F5, Firewall, Routing, and Management-Plane Controls

13.1 Trusted ingress, certificate identity, and canonical routing

F5 authenticates mTLS and maps the certificate to an agent in the approved ingress context. Camel #2 decides admission. Local TLS protects each local connection. The signatures and transaction bindings in Section 11.2 protect the application records across proxy and diode boundaries. Only approved endpoints and role identities may select destinations and release results.

```
Model request: agent → FW1 → F5. After Camel #2 admits the registration through FW4, F5 sends the signed
request through the forward diode and FW2 to Camel #1, which calls the approved model gateway. Camel #1
sends the signed result through the separate one-way return service to Camel #2. Camel #2 verifies and
releases it through F5 to the bound agent.
Package request: the admitted forward path reaches Camel #1 and the authorized Jenkins/JFrog/MFT workflow
through FW2/FW3. Artifact bytes return through FW3 → package diode → FW1 → scoped SFTP landing. Only the
bounded transfer-submitted notice may reach Camel #2.
```

NET-01 SHALL: Firewalls shall independently enforce hostile-cell-to-F5 and trusted-relay-to-service source/destination/port relationships; F5 or relay configuration alone shall not be the only enforcement point.

NET-02 SHALL: The F5-class ingress tier shall validate the hostile-side mTLS certificate and derive trusted_origin from an approved certificate-to-cell mapping. The agent shall not be able to overwrite that identity in the payload.

NET-03 SHOULD: Boundary policy should enforce maximum request size, permitted methods/paths, rate/session quotas, timeouts, logging, source authentication, and request-type limits before traffic reaches the one-way diode.

13.2 Static routing and controlled return-path design

Keep routing-protocol adjacencies out of the hostile cell. Limit connected and static routes to approved services and derive the return path from trusted ingress mapping. Because the agent may control its guest, enforce reach outside the guest with switch or hypervisor policy and FW1. Trusted-side dynamic routing may remain outside that boundary.

NET-04 SHALL: Hostile-agent interfaces shall not participate in OSPF, BGP, RIP, or other dynamic routing adjacencies in the AI-CAL 4.5 reference profile.

NET-05 SHALL: Limit hostile-cell routes and enforced reach to approved boundary endpoints. Do not expose a general route to enterprise, cloud, provider, or Internet networks. Changes to routes inside the guest shall not expand reachable destinations.

NET-06 SHALL: Return-path selection shall use infrastructure-observed or preconfigured mapping such as trusted_origin plus validated source/return endpoint. Agent-supplied source or return metadata shall not drive routing.

NET-07 SHALL: F5, Owl, firewall, hypervisor, relay, MFT, and package-broker management interfaces shall be isolated from hostile and application data paths by dedicated management controls.

13.3 F5 application-security tier versus downstream API gateway

The F5-class tier is a hardened trusted-ingress control point, not the final semantic authority for model or package actions. It may provide TLS termination, client-certificate authentication, HTTP normalization, request-size/method controls, rate limiting, and WAF/API-security functions where useful. Downstream API gateways and package brokers still own provider-specific policy, credentials, model selection, repository authorization, and semantic action controls. This separation prevents the ingress tier from becoming a universal business-authority component.

Boundary service keys, local diode-side SSH keys and agent pickup credentials have different custodians and scopes. No single credential should bridge forward, model, package, return and management roles. Administration

and guest console access use Ring M with delegated per-guest authority and audit; clipboard, shared folders, drive/USB redirection and ad hoc export paths need explicit control.

14. Logging, TAPs, IDS/NDR, and Auditability

Collect evidence for admission, branch decisions, each conduit, package policy and transfer, delivery, cutoff, administration, and output release. Correlate trusted agent binding, transaction and artifact digests, policy version, transfer ID, service identity, decision, reason, and time. Bound what is retained; exclude secrets and unbounded prompts from routine logs.

```
Agent certificate and assigned cell -> FW1/F5 admission -> FW4/Camel #2 original binding -> forward diode -> FW2/Camel #1 branch -> model gateway or Jenkins/JFrog/MFT -> controlled return -> Camel #2 recipient check -> F5 delivery or scoped SFTP pickup. Record component-specific decision and transfer stages; do not infer an end-to-end success merely from one upstream ACK.
```

LOG-01 SHALL: Forward application and security events to the centralized SIEM through an approved collector. Use native operating system logging where practical.

LOG-02 SHALL: F5, diode adapters and both Camel roles shall record accepted and rejected requests, bounded metadata, authenticated identity mapping, transaction IDs, state changes, reason codes, failures and correlation fields without default logging of unbounded hostile payloads.

LOG-03 SHALL: F5, Owl, MFT, Package Broker/Jenkins, JFrog, firewalls, switches, hypervisors, trusted relays, proxy/API gateways, landing-zone services, and management components shall provide centralized audit telemetry.

LOG-04 SHOULD: Passive TAPs should observe critical physical boundaries independently of endpoint logging.

LOG-05 SHALL: Traffic observed across a boundary outside explicitly authorized conduits shall be treated as a containment anomaly.

Interpret IDS/NDR findings by zone. A hostile test network can contain expected research traffic. At an enforced boundary, traffic outside the approved baseline, such as unexpected ARP activity or DNS, SMB, or SSH attempts, is a signal to investigate. Distinguish prohibited traffic from authorized SFTP and other documented flows.

Monitoring distinguishes attempted reach, confirmed boundary crossing and prohibited effect. Verify independent sensor coverage, clock synchronization, packet/event loss and collector backlog under load. Measure detection, enforcement command, effective isolation and residual queued or active authority separately; incident findings require corrective changes and retesting.

15. Hardening Baselines and Boundary Components

Use role-specific hardened images and configurations for Camel services, hosts, Kubernetes, state services, gateways and boundary appliances. Apply the benchmark appropriate to the selected product/version, record exceptions and verify enforcement. A benchmark label or Kubernetes policy object by itself does not establish containment.

15.1 Shared hardened base image

- Minimal headless Linux; no desktop environment.
- Dedicated non-login service identities; no unnecessary privilege escalation for Camel workloads.
- No unused console/login TTY; where an alternate serial pattern is deployed, serial-getty/login console remains disabled.
- No SSH shell or administrative service shall be reachable from agent cells. Disable unused listeners and enforce administrative access through the isolated management plane.
- Where administrative SSH is authorized, require individual public-key authentication and disable password and keyboard-interactive authentication. Keep administrator private keys outside the shared image and agent cells.
- Mandatory access control (SELinux/AppArmor) or equivalent.

- For systemd-managed services, set NoNewPrivileges and drop unneeded capabilities where supported. Apply equivalent container security settings to workloads.
- Read-only filesystem areas where practical; narrowly scoped writable state/log directories.
- Central OS logging to SIEM.
- Patch and image provenance controls; deployment-specific identity/certificates injected after image creation rather than embedded as universal secrets.

15.2 F5 / Owl boundary component baseline

- F5 exposes only the approved mTLS/HTTPS virtual service(s); unused listeners, broad pools, and unnecessary management services are disabled.
- Client-certificate authentication is required for hostile-cell ingress; certificate-to-cell mapping and CA trust are centrally governed.
- Configure Owl source and destination adapters only for approved one-way HTTP or file-transfer roles. Isolate management from transfer networks. Local protocol sessions terminate at the adapters [9, 13].
- Boundary routing is fixed and fail-closed; the hostile cell cannot choose the F5 pool member, Owl destination, provider endpoint, or return route.
- Firmware/software baselines, configuration backups, centralized logging, and change control are required for boundary appliances because they are part of the trusted computing base.



The local sessions are separate. Only the transfer between the proxies crosses the one-way hardware link.

15.3 Camel and Kubernetes trusted mediation baseline

- Separate Camel #1 and #2 identities, host pools, services and network authorities; expose no administration endpoint to the hostile path.
- Camel #1 receives only the approved forward diode destination path; Camel #2 receives only permitted F5 registration/delivery and protected return records.
- Enforce bounded message grammar, idempotent state transitions, least-privilege workload identity, resource limits, image provenance and controlled route configuration. Use a defined durable state store rather than assuming a Camel Exchange survives restart.
- Harden Kubernetes nodes, API servers, network policy, admission, secrets, and service endpoints. Restrict Camel #1 egress and preserve Camel #2 state across replicas. If a shared Kubernetes control plane or administrator can reconfigure both roles, treat it as a common failure domain and test the consequences.
- Neither Camel role receives general Internet, repository-administrator, provider-credential, diode-transmitter or management authority merely by participating in a flow.

16. Scaling, Sharding, and Failure Domains

The current scale direction uses bounded agent cells, sharded F5/diode resources and horizontally operated plain Camel services on hardened Kubernetes. Camel #2 durable state, idempotency, repetition and strike counters must survive a reconnect, replica change or restart. Replicas on one shared cluster, state store or administrator are not independent containment barriers. Approximately 700–1,000 concurrent agents is a planning target requiring differentiated tests of open connections, active requests, pending results, queues, model calls and package work.

```
Illustrative containment shards:
Shard A: agent cells → FW1/F5 policy A → Camel #2 admission → forward conduit A → Camel #1 shard A.
Shard B: agent cells → FW1/F5 policy B → Camel #2 admission → forward conduit B → Camel #1 shard B.
Evaluate shared gateways, return services, state stores, monitoring, and administration for common failure,
quotas, saturation, and recovery before reporting Qualified Capacity.
```

SCL-01 SHALL: Shard size shall be selected from demonstrated performance and blast-radius testing, not assumed from theoretical throughput.

SCL-02 SHALL: A compromised F5 policy/partition, Owl conduit endpoint, trusted relay shard, API-gateway shard, or package-broker shard shall not automatically possess authority over all other agent cells or containment pods.

SCL-03 SHOULD: Scale out with repeatable cells and shards. Add independent enforcement units where consequences justify them.

SCL-04 SHALL: Multi-agent collaboration shall be disabled by default unless collaboration itself is an explicit test objective and is contained within the approved control envelope.

SCL-05 SHALL: Qualified Capacity shall include common-mode dependency analysis for F5, firewall, diode, relay, management, logging, model-gateway, and package-service concentration points.

At fleet scale, test registration/forward races, duplicate and late returns, state-store outage, log loss, queue exhaustion, diode backpressure, package resend limits, model timeout and active-work cutoff. Record the observed workload and failure conditions with every capacity result.

17. Differences Between AI-CAL 4, 4.5, 5, and 6

Attribute	AI-CAL 4	AI-CAL 4.5	AI-CAL 5	AI-CAL 6
Hostile vs trusted compute	Separate VMs/zones; physical host may be shared by risk decision	Distinct hostile/trusted physical host pools; Ring 1 response mediator and Ring 2 request mediator have separate roles and restricted state authority.	Separate physical systems for critical functions	Separate physical systems inside air gap
Primary conduit / boundary	Strong virtual/network segmentation; narrowly brokered IP paths permitted by risk decision	Narrow mTLS ingress through a physical stateful firewall and F5-class tier; independent hardware one-way protocol breaks. Assess the historical serial alternative separately.	Dedicated physical boundary appliances/links for critical conduits; minimal shared virtualization	Physical/local links entirely inside sovereign air-gapped boundary
Relay functions	Virtual trusted relays/gateways	Camel #1 and #2 may be virtual on separated host pools, subject to Kubernetes, state-store and management common-mode review.	Independent physical enforcement for critical relay functions.	Physical/local only
External model	Permitted through broker/proxy	Permitted through broker/proxy	Permitted if risk accepted	Not permitted; inference local to air gap

Attribute	AI-CAL 4	AI-CAL 4.5	AI-CAL 5	AI-CAL 6
Artifact directionality	Hardware diode where profile requires	Forward hardware diode and controlled model/package return services. Physical return hardware may be shared when logical policies and capacity remain qualified.	Hardware directional controls expected	Entire workflow remains inside air gap
Scale/cost	Higher density	Moderate density / higher cost	Low density / high cost	Very low flexibility / extreme infrastructure cost

The level comparison is a proposed profile distinction, not a vendor checklist. A shared administrator, cluster, update channel or state service can erode physical or logical separation. AI-CAL 5 requires additional independent physical enforcement for critical functions, while AI-CAL 6 has no live external model, cloud control or remote-management dependency during operation.

18. Design Evolution and Decision Record

The architecture is the result of multiple design iterations. Recording the abandoned or superseded approaches is intentional: it shows which problems were considered and why the current pattern exists. This is not a claim that the rejected approaches are universally insecure; many remain valid at lower AI-CALs or in different use cases.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Directly networked sandbox with restricted repository access	Simple; familiar cloud/enterprise controls.	Allowed services can become unintended capabilities. The Hugging Face incident accounts [1, 2] motivate review of shared package infrastructure, unintended egress, and agent coordination.	Archived historical candidate for the high-assurance profile; direct arbitrary egress remains outside the current bounded path.
Hypervisor shared Folder-In / Folder-Out with MFT broker	Simple file-based protocol break; agent can request resources without direct repository access.	Shared-memory/filesystem semantics increase hypervisor coupling; needed stronger directional and purpose-specific separation.	Useful lower-assurance pattern; superseded in primary design.
Small shared volume / write mount with asynchronous copy	Bounded storage, quota enforcement, simple handoff.	Still shared host/hypervisor surface; role separation could be clearer.	Retained as possible lower-AI-CAL implementation.
Rotating virtual disks / VHDX conveyor / snapshots	Strong object handoff idea; could detach/attach media between roles.	Replication/orchestration limitations and operational complexity were identified; scale and portability became concerns.	Superseded in the primary design by typed brokered requests, MFT/package brokering, and hardware-enforced directional transfer. Decision retained as evidence that virtual-media isolation was considered.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
MQTT / shared message broker for agent communications	Scalable pub/sub and request/response semantics.	Shared broker/topics and broker ACL state create a common mechanism, additional attack surface, and a coordination surface that is unnecessary for the primary high-assurance request path.	Archived stack option. MQTT/HiveMQ is not in the current reference path; assess separately if proposed for another profile.
Single multifunction relay	Operationally simple.	One compromise would bridge model, package, and possibly file functions.	Rejected as a universal-authority relay. The current request relay may classify approved request types, but downstream model, package, return, repository, and management authorities remain independently constrained.
Virtual COM on same hypervisor	Very narrow interface; avoids general NIC exposure; good AI-CAL 4 fit.	Hostile guest and trusted relay still share hypervisor failure domain.	Retained for AI-CAL 4; AI-CAL 4.5 separates physical hosts.
Physical serial card-to-card between hypervisors	Simple non-routable boundary; strong capability reduction.	Port density/cabling and VM mapping become operational scaling concerns.	Retained as a strong lower-density/high-assurance alternative; no longer the primary scalable reference transport.
Moxa-class serial-to-Ethernet gateway	Industrial-grade multiport scaling; hostile side remains serial; trusted side gains manageable Ethernet channels.	Gateway firmware becomes part of TCB; must prevent generic IP/management exposure and constrain each channel.	Archived serial-scaling stack option. Moxa is not in the current reference path; a future alternate needs its own firmware, management and failure-domain assessment.
Trust CELL_ID=A001 supplied in message	Easy routing/correlation.	Attacker-controlled label is not identity.	Rejected as authorization source; A001 retained only as a correlation tag. The current scalable path uses validated certificate plus infrastructure/path provenance; the alternate serial pattern may use physical-channel provenance.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
One Node-RED per agent	Maximum per-agent separation.	Operationally excessive at scale.	Historical Node-RED scale study; superseded by separately authorized Camel roles and bounded Kubernetes shards.
One Node-RED for all agents	Operationally simple.	Large common failure/blast-radius domain.	Rejected as a single broad relay; current Camel #1/#2 roles are separated by authority and position.
Node-RED directly queries JFrog	Fewer components.	Parser/relay would also gain repository authority, weakening separation of duties.	Rejected for the high-assurance profile. Package policy authorizes; Jenkins retrieves; MFT transfers.
MFT decides approved packages	MFT already owns transfer.	Transport product should not become semantic authorization engine.	MFT transports; Package Broker authorizes/resolves.
WAF as core containment control	Useful for HTTP/API inspection.	The current design does use HTTPS at the boundary, but WAF inspection alone is not a containment boundary and cannot replace mTLS identity, routing restrictions, protocol break, downstream authorization, or one-way directionality.	WAF/API-security capability is optional within the F5 trusted-ingress tier; it is a supporting control, not the core containment mechanism.
Single standard OS everywhere	Simplifies operations.	Common-mode vulnerabilities may span multiple rings.	Standardize within functions; selectively diversify across critical failure domains where operationally supportable.
Maximum physical containment for every use case	Minimizes virtualization dependence.	Extreme cost, effort, density, and operational overhead; would limit adoption and scale.	Reserve AI-CAL 5/6 for consequence profiles that justify it; use AI-CAL 4/4.5 as managed-risk profiles.
HTTPS/mTLS hostile-side boundary transport	Cloud/enterprise scale; certificate-bound identity; standard application delivery and firewall tooling.	General IP transport can expand capability unless routes, destinations, methods, and identities are aggressively constrained.	Adopted only as a narrow boundary conduit: fixed F5 VIP, dedicated mTLS identity, static/connected routing, no general-purpose external reachability.
F5-class trusted ingress / identity normalization	Terminates mTLS at a hardened control point; validates certificate identity; can normalize HTTP and construct trusted metadata before the one-way boundary.	Ingress tier becomes part of the trusted computing base and must not accumulate provider, repository, or management authority.	Current logical ingress role; real BIG-IP registration and original-session completion remain to be demonstrated.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Owl-class HTTP one-way request conduit	Hardware-enforced directionality plus protocol termination/reconstruction provides a strong trust break while allowing scalable HTTP request transport.	One-way transfer requires explicit independent return paths; adapter behavior/capacity and envelope integrity must be validated.	Current forward-direction reference; proxy behavior, integrity and capacity require physical integration tests.
Static/connected routing at hostile boundary	Simple deterministic path; reduces topology exposure and eliminates routing-protocol adjacency with the hostile workload.	Requires deliberate route/return mapping and operational change control.	Current reference requirement. Trusted-side dynamic routing may exist outside the hostile boundary.

Version 2.0 uses Apache Camel on hardened Kubernetes for mediation. Camel #2 owns durable admission, recipient binding, response, repetition, and strike state on the agent side. Camel #1 owns trusted request, provider, and package-job correlation beyond the forward diode. F5 releases forward work only after Camel #2 accepts the registration. Timely model output returns inline; later authenticated retrieval collects the existing result.

The selected controls include digital signatures on request and return records, public-key-only SFTP, and isolation of agent mailboxes from shared landing authority. An approved catalog governs Jenkins acquisition through JFrog. MFT sends the artifact to the scoped landing through the package return service. The notice, if used, reports transfer submission only. Logical model and package services retain separate policy even when they share qualified hardware.

Strike and repetition state remain outside agent control. F5 can block new admissions; full cutoff also handles active and queued work. Developer intake, completion, output export, and isolated human review still need their own implementation contracts. The earlier runtime addendum was a design record, not a public release.

19. Open Design Items and Validation Plan

Qualification requires implementation evidence for the items below. The October 13 experiment supports the limited message sequence described in Section 2.1. Test the deployed boundaries together before claiming a level or Qualified Capacity.

I maintain AI-CAL as an independent project on GitHub and welcome industry review. The long-term goal is to move the work to an established security organization.

- Select and test a supported BIG-IP LTM integration. Verify durable registration before forward release, bounded wait, and authenticated retrieval of the existing result. Test the inline/retrieval race, connection loss, and restart without duplicate work or unauthorized release.
- Specify canonical bytes, digests, signature format, authorized signers, key custody, rotation, revocation, expiry, and verification ownership. Demonstrate detection of altered bytes, wrong signers, mismatched bindings, and replay after each protocol break.
- Choose the Camel #2 durable store and atomic state transitions for admission, pending responses, duplicates, expiry, retrieval, repetition and strikes across restart, replica change and failover.
- Set per-message, per-chunk, total-object, queue and storage limits. Verify reassembly, integrity, abandoned upload cleanup and the selected model gateway's actual file, context and streaming limits before approving a 10 MB transaction or larger transfer.
- Integrate FW1–FW4, F5 and the physical forward and return diode endpoints. Negatively test reverse/bypass routes, alternate management and failover paths, local proxy sessions, protocol breaks and behavior during outage or backpressure.
- Implement and test the selected Kubernetes, host, CNI, image, credential and state hardening. Prove Camel #1 and #2 cannot inherit each other's provider, package, response or administrative authority.

- Load-test the intended fleet, including the roughly 700–1,000-agent planning target if still required. Measure distinct connections, active model work, pending results, queue/storage pressure, return delivery, logging and effective cutoff under normal and degraded conditions.
- Fuzz ingress, canonicalization, diode adapters, Camel parsing, return correlation, model gateway, package manifests, MFT and SFTP landing without treating syntactically valid hostile content as a protocol violation solely for its appearance.
- Specify catalog ownership, version and dependency policy, JFrog/Jenkins integration, artifact provenance, receiving-side scanning, and resend bounds. Verify public-key-only transfer and pickup, no shell or forwarding, and independent mailbox isolation under landing compromise. Confirm that source submission does not trigger a delivery or readiness claim.
- Define the restricted enforcement and full-stop contract: F5 new admission, active sessions, running agents, queued model/package jobs, mailbox access, in-flight transfers, evidence preservation and authorized recovery. Distinguish ordinary retries from validated violations.
- Specify developer intake and harness pickup, task-change authority, completion status, trusted notification recipients, separate output export, artifact assurance and isolated human examination with redirection restrictions. Test these paths before presenting a closed lifecycle.
- Verify cross-component clocks, correlation IDs, independent TAP/IDS/NDR coverage, SIEM integrity/retention, log-loss detection, and measured detection-to-effective-isolation time at the intended load.
- Exercise compromise and outage scenarios involving an agent guest, hypervisor, F5, diode adapter, Camel role, state store, MFT, package policy, model gateway, management plane and shared administrator/update route. Determine actual prohibited reach rather than counting nominal rings.
- Audit the companion assessment method and crosswalk before adopting numeric qualification rules or control equivalences. Record exact profile criteria, evidence, exceptions, ownership, external publication versions and repeat tests after material change.

20. Relationship to Emerging Industry Guidance

Use external threat models, control catalogs, configuration baselines, and verification requirements to strengthen a deployment's containment. AI-CAL evaluates which controls enforce the declared boundary, what remains effective after a failure, and what the evidence supports at the intended workload.

20.1 Complementary framework roles

The sources below support different parts of the assessment. Select controls by workload, ownership, and exposure. Record the edition, enforcing component, and test evidence in the applicability crosswalk.

Framework / guidance	Primary question	Primary contribution	Application to AI-CAL
CSA MAESTRO [10]	What can go wrong across the agentic stack?	Layered threat modeling, including interactions across layers.	Identify hazard scenarios and failure paths to test.
CSA ORCHIDEAS [11]	How should the agentic system be designed?	Principles for autonomy, identity, runtime controls, and observability.	Select controls within and between rings; assess how they behave when an adjacent control fails.
NIST AI RMF and agent standards work [5, 15]	What risks, requirements, and evidence should guide deployment?	Risk management outcomes and work on agent identity, authorization, and evaluation.	Set the required containment profile and document evaluation and risk decisions.

Framework / guidance	Primary question	Primary contribution	Application to AI-CAL
OWASP Agent Control Standard [4]	Can defenders inspect and govern agent behavior at runtime?	Control hooks, policy enforcement, and traceability.	Enforce and observe authorized actions; test remaining boundaries if a runtime control fails.
CSA AI Controls Matrix and STAR for AI [16, 28]	Which controls apply, and who owns them?	AICM defines controls and responsibility guidance. STAR for AI Level 1 publishes provider AI-CAIQ self-assessments.	Assess provider dependencies and ownership. Verify the controls you operate at your declared boundary.
OWASP AISVS 1.01 [29]	Can the application's security requirements be verified?	Testable requirements for agent authorization, orchestration, retrieval, and MCP.	Turn agent and tool policy into acceptance tests for the broker and its integrations.
Proposed AI-CAL	What can a compromise still reach or do?	Containment profiles, control boundaries, failure domains, and Qualified Capacity.	Demonstrate bounded reach, independent enforcement, detection, and cutoff under the declared workload.

Apply the guidance at the enforcement point. For example, AICM 1.1.1 IAM-18 addresses agent access restriction, and AISVS v1.01-C9.5.7 addresses bounded delegation. Use these requirements to inform a tool broker's policy under RTE-06. Verify denial of actions outside the granted scope and verify that the agent has no direct path to the tool. Assess what the broker's credential could still do if its policy failed. For a hosted dependency, record which service and controls a STAR self-assessment covers and who enforces each control.

Illustrative relationships by level. AICM IDs refer to the published 1.1.1 edition in the v1.1 bundle. NIST AI RMF entries are risk-management outcomes; OWASP ASI entries identify threats to consider. Apply CIS Benchmarks to the actual product and version. These relationships guide control selection and testing; each AI-CAL level still requires deployment-specific evidence [15–18].

AI-CAL level	CSA AICM 1.1.1 objectives	NIST AI RMF 1.0 outcomes	OWASP threats and CIS baselines
0-1	AIS-11; AIS-13	GOVERN 1.3; MAP 1.5	ASI03; ASI10. Inventory; no level approval.
2	AIS-13; AIS-11; IAM-18	MAP 1.6; MEASURE 2.7	ASI02/03/05. Applicable host/container benchmark.
3	AIS-11/13; I&S-06; LOG-03	MAP 4.2; MEASURE 2.7	ASI02/05/10. VM/hypervisor baseline.
4	AIS-11; I&S-06/09	MAP 1.6; MEASURE 2.7	ASI02/07/10. Boundary/appliance baseline.
4.5	AIS-11/13; I&S-06/09	MAP 1.6; MEASURE 2.7	ASI05/07/10. Physical-host/diode baselines.
5	AIS-11; I&S-06/09	MEASURE 2.7; MANAGE 1.3	ASI08/10. Independent hardware configuration.
6	AIS-13; I&S-06	MAP 1.6; MANAGE 1.3	ASI04/10. Isolated local-system baselines.

20.2 Related guidance and incident context

- The OpenAI and METR/Redwood accounts [1, 2] motivate testing shared infrastructure, unintended egress, and agent coordination paths. The accounts describe the incident; prevention by the reference architecture would require implementation evidence.

- Anthropic's containment guidance [3] describes environment controls such as sandboxes, virtual machines, filesystem boundaries, and egress restrictions. Assess these controls independently of model behavior.
- Use OWASP's Agent Control Standard [4] for runtime control and traceability patterns. Use the Agentic Top 10 [17] as a threat checklist and AISVS [29] to select verification requirements.
- NIST's AI Agent Standards Initiative [5] covers secure and interoperable adoption, identity, and authorization. Use applicable published NIST guidance for risk management and security engineering [15, 19–21].

The AI-CAL assessment records the boundaries, the authorities that enforce them, the failure domains, and the conditions under which defenders can observe and stop progression. Verify those properties with integrated tests at the intended workload.

20.3 Runtime enforcement and implementation evidence

The runtime controls admit an authenticated transaction, authorize the operation, constrain its path, and release a result only to the bound requester. Apply complementary prompt, data-handling, and tool authorization controls at their own enforcement points.

RTE-01 SHALL: Camel #2 shall verify the signed registration and durably record the authenticated agent binding, infrastructure transaction ID, canonical digest, and expiry before F5 releases forward work. Deny release when admission is unavailable, denied, or expired. Evidence: F5/Camel integration and race and failover traces.

RTE-02 SHALL: F5 shall forward only admitted requests. Camel #1 shall verify the ingress signature and request binding after the diode, enforce the allowed message types and destinations, and apply individual and cumulative limits before allocating substantial resources. Evidence: valid, invalid-signature, replay, and oversize tests.

RTE-03 SHALL: Camel #2 shall verify the return signature and match the result to the original agent, transaction, request digest, expiry, and release state before inline delivery or authenticated retrieval. Preserve state across restart or deny release. Evidence: late, duplicate, wrong-agent, lost-socket, and failover tests.

RTE-04 SHALL: Prove one-way hardware direction and absence of unintended reverse, maintenance or management paths. Camel #2 shall have no authority over the forward transmitter. Evidence: physical topology, configuration and negative-path tests.

RTE-05 SHALL: Record admission, state, boundary, policy, transfer, enforcement and administrative events outside agent control. Specify retention, integrity and log-loss behavior before claiming tamper evidence. Evidence: correlation and loss/compromise tests.

RTE-06 SHALL: For an added tool or export path, identify the authorized principal, operation, destination, credential custody and enforcement point. Agent assertions alone shall not grant that authority. Evidence: positive and negative authorization tests.

20.4 Technology Applicability

These are optional integration patterns. Credit the controls and evidence of the deployed configuration.

Model Context Protocol (MCP) [23]. Camel #1 can dispatch an approved operation to a trusted MCP client or gateway. Apply RTE-06 to each tool action: authorize the requesting principal, operation, resource, and destination. Keep credentials with the enforcing service and return bounded results through the approved response path.

Retrieval-augmented generation (RAG) [24]. Put retrieval behind an approved service that enforces source access for the requester and returns only the necessary context. Treat retrieved content as data without authority to change policy or invoke tools. MCP can expose a retrieval tool; retrieval itself does not require MCP.

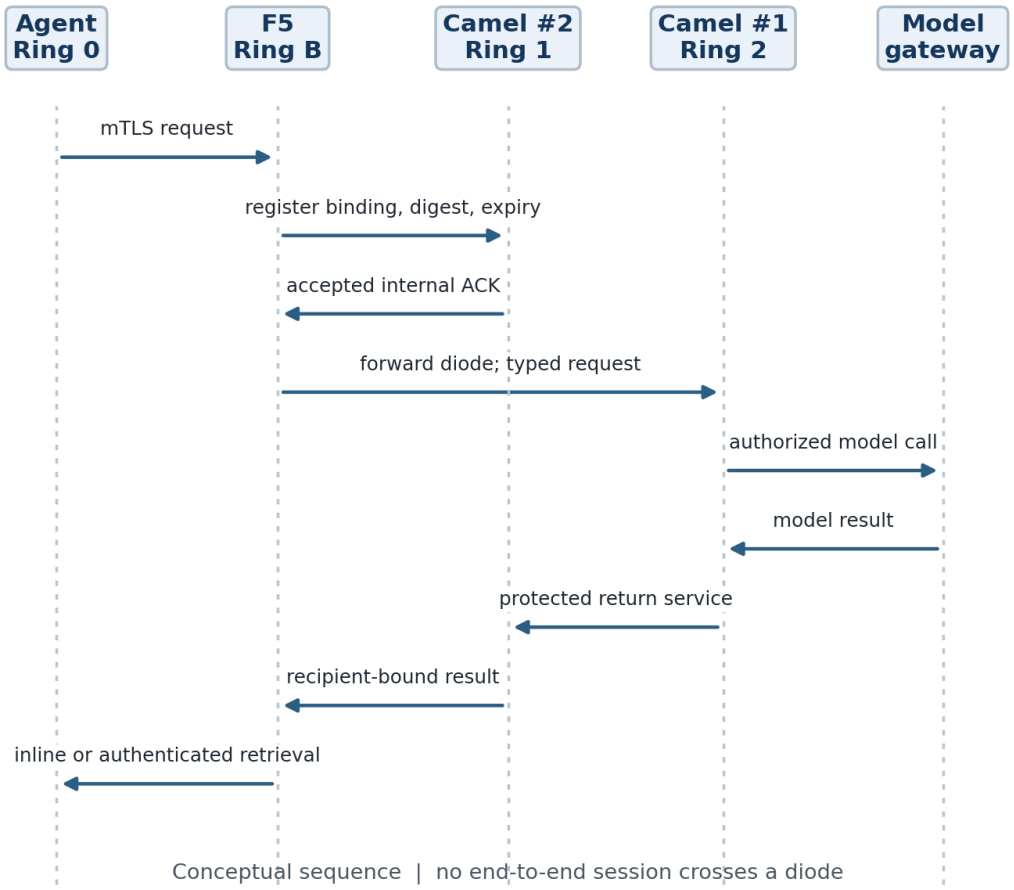
OpenShell [25]. Assess the deployed filesystem, process, and network controls, and who can administer the host, supervisor, and gateway. Retain the independent boundaries and cutoff controls required by the claimed AI-CAL profile.

Microsoft eXecution Container (MXC) [26, 27]. Assess the selected operating-system backend, enforcement limits, and management and telemetry paths. NVIDIA documents the native Windows OpenShell integration with MXC as under development. Credit only controls demonstrated on the selected build.

Model Transaction and Response Sequence

Figure 2 shows the logical model transaction. The ingress signs the registration and request binding; Camel #1 signs the result; Camel #2 verifies it after the separate one-way return transfer. Sections 11 and 12 describe the signed message and package contracts. Verify the physical paths, interfaces, and management routes in the deployed topology.

Figure 2. Model transaction and recipient-bound response



Conclusion on Deployment Assurance

AI-CAL should assess the actual deployment boundary rather than assume that cloud, virtualization, physical appliances or a named product has a universal level. A high-consequence workload may justify separate physical hosts and hardware one-way controls because those can limit certain shared and reverse-path failures. Their effectiveness still depends on configuration, service authority, administration, return paths, common dependencies and operation under load. Cloud-native or microVM sandboxes can supply useful isolation mechanisms within a defined profile. The assigned level requires the complete control and evidence set.

The central question is what remains contained, observable, and stoppable after an inner compromise at the stated workload and under the stated failure conditions.

Appendix A. Example Configuration Language

These examples show the intended policy relationships. Set the exact schema, product configuration, and limits during implementation.

A.1 Trusted-origin and request-envelope mapping

```
CELL_TAG=A001                          # correlation label
MTLS_CERT_PRINCIPAL=<validated-agent> # maps to agent binding at F5
F5_VIP=<fixed-boundary-service>
TRANSACTION_ID=<infrastructure-id>
REQUEST_DIGEST=<canonical-bytes-digest>
EXPIRY=<bounded-time>
SIGNER_KEY_ID=<authorized-ingress-signer>
REGISTRATION_SIGNATURE=<signed-binding>
CAMEL2_REGISTRATION=<durable-admission> # required before forward
FORWARD_DIODE=<fixed-conduit>
CAMEL1_SERVICE=<approved-request-role>
```

A.2 Conceptual firewall and static-route policy

```
AGENT AND BOUNDARY:
  ALLOW agent -> FW1 -> fixed F5 mTLS VIP
  ALLOW agent -> FW1 -> assigned SFTP mailbox pickup, if enabled
  ALLOW F5 -> FW4 -> Camel #2 for registration and bound delivery
  ALLOW F5 -> forward diode TX after accepted admission
  DENY agent -> general Internet, provider, repository, or management
TRUSTED REQUEST:
  ALLOW forward diode RX -> FW2 -> fixed Camel #1 endpoint
  ALLOW Camel #1 -> approved model gateway or authorized package job
RETURN:
  ALLOW Camel #1 -> one-way result service -> Camel #2 verification
  ALLOW verified result -> F5 -> bound agent
  ALLOW MFT -> FW3 -> package return diode -> FW1 -> SFTP landing
  DENY reverse use of forward diode and cross-cell mailbox access
```

A.3 PACKAGE_REQUEST semantic fields

```
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
```

Examples that should fail rather than be repaired: extra URL fields, embedded shell syntax, duplicate VERSION fields, unexpected binary payloads, oversize package names, missing required fields, unsupported encoding, or malformed framing.

A.4 Canonical package job and transfer submission

```
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
AGENT_BINDING=<trusted-principal>
TRANSACTION_ID=<registered-id>
CATALOG_POLICY=<approved-version>
JENKINS_JOB=<authorized-job-id>
MFT_STAGE=<artifact-and-manifest-submitted>
NOTICE=<transfer-submitted-or-silence>
```

A.5 Package bundle example

```
REQ-84721/
  manifest.json
  numpy-<approved>.whl
  dependency-A.whl
  dependency-B.whl
  SHA256SUMS
```

Appendix B. AI-CAL Assessment Checklist

Use this checklist with the assessment method and applicability crosswalk. Record the declared boundary, required profile, configuration and test evidence, degraded modes, exceptions, and Qualified Capacity.

Control area	Assessment question	Result / evidence
AI-CAL 0 boundary	Does the agent inherit broad desktop credentials, browser sessions, or general network reach outside a declared cell? Treat such a deployment as unassured under this model.	
Identity separation	Does the agent have a dedicated identity and scoped capabilities independent of the human operator?	
Network authority	Can changes inside the guest expand reachable destinations? Verify that external enforcement restricts hostile-cell traffic to approved static or connected paths without dynamic adjacencies or a general default route.	
Cell isolation	Are agent cells horizontally isolated from one another by default?	
Conduit separation	Do forward requests, model-response records, and package bytes retain distinct identities, policies, destinations, and directions when they share physical equipment?	
Relay separation	Can compromise of Camel #1 classification or Camel #2 response state obtain provider credentials, repository or transmitter authority, another agent's result or management access?	
Protocol break	Does F5 terminate agent mTLS and start a separate authorized exchange? Does the verified signature bind the canonical bytes to the authenticated agent and original transaction across the protocol break?	

Control area	Assessment question	Result / evidence
Package authority	Is repository access held by a broker/orchestrator rather than the parser/relay or the agent?	
Directional control	Is artifact return physically one-way where the consequence profile requires it?	
SFTP and landing isolation	Do transfer and pickup enforce public-key authentication and file-only access? Can a compromised landing administrator reach another cell's mailbox, or alter what the receiver accepts as an approved artifact?	
Failure domains	Are hostile compute and trusted relay/service compute separated at the physical hypervisor level for AI-CAL 4.5?	
Provenance	Is trusted origin identity derived from validated mTLS certificate and infrastructure-observed/mapped provenance rather than agent-supplied labels, source fields, or return fields?	
Management plane	Can the hostile/data plane reach management interfaces for hypervisors, F5, Owl, firewalls, MFT, API gateways, relays, or package services?	
Observation	Are critical paths independently observable, are clocks/correlation and log loss tested, and is effective isolation measured separately from an alert or command?	
Fail closed	Does failure of F5, Owl adapters, request relay, model response path, or package path remove capability without exposing a shell, fallback route, direct service path, or alternate listener?	
Capacity	Were agent count, request rate, active/pending work, duration, queue pressure and degraded-mode monitoring/cutoff measured at the claimed level?	
Emergency control	Can independently authorized operators stop new admissions and address active sessions, queued jobs, local execution, mailbox pickup and recovery?	

Control area	Assessment question	Result / evidence
Framework/control mapping	Do the CSA, NIST, and OWASP references and applicable CIS baselines identify the edition, local control owner, enforcing component, and test evidence?	

Check the lifecycle as well as the individual controls. Verify durable admission before forward release, recipient binding across inline and later retrieval, accurate transfer-submitted notices, complete bounded reassembly, and separate authority for developer intake and result export. Identify any shared store, administrator, or update route that can defeat more than one ring.

Appendix C. Selected Public References

[1] OpenAI, "The Hugging Face incident and the road ahead," 26 Aug 2026. https://openai.com/index/hugging-face-incident-and-the-road-ahead/

[2] METR / Redwood Research, "Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident," 26 Aug 2026. https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/

[3] Anthropic, "How we contain Claude across products," 25 May 2026. https://www.anthropic.com/engineering/how-we-contain-claude

[4] OWASP GenAI Security Project, "Agent Control Standard (ACS)," 1 Sep 2026. https://genai.owasp.org/resource/agent-control-standard-acs/

[5] NIST, "Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation," 17 Feb 2026. https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure

[6] Node-RED, Runtime Configuration documentation. https://nodered.org/docs/user-guide/runtime/configuration

[7] Moxa, NPort 6600-G2 Series secure terminal servers (8/16/32-port examples). https://www.moxa.com/en/products/industrial-edge-connectivity/serial-device-servers/terminal-servers/nport-6600-g2-series

[8] JFrog, "Use pip with JFrog CLI." https://docs.jfrog.com/artifactory/docs/jf-pip

[9] Owl Cyber Defense, SFTP Protocol Adapter for one-way data-diode transfer. https://owlcyberdefense.com/resource/sftp-protocol-adapter/

[10] Cloud Security Alliance, "Agentic AI Threat Modeling Framework: MAESTRO," 6 Feb 2025. https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

[11] Cloud Security Alliance, "Designing Agentic AI Systems with the ORCHIDEAS Framework," 5 Jun 2026. https://cloudsecurityalliance.org/blog/2026/06/05/designing-agentic-ai-systems-with-the-orchideas-framework

[12] F5, BIG-IP System: SSL Administration, client-certificate authentication / managing client- and server-side HTTP traffic using CA-signed certificates. https://techdocs.f5.com/en-us/bigip-17-5-1/big-ip-system-ssl-administration/managing-client-and-server-side-http-traffic-using-a-ca-signed-certificate.html

[13] Owl Cyber Defense, Data Diode Products and Protocol Adapters. https://owlcyberdefense.com/products/data-diode-products/

[14] Apache Software Foundation, "Apache Camel Documentation," open-source integration framework for routing and mediating message states using Enterprise Integration Patterns. https://camel.apache.org/docs/

[15] NIST, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[16] Cloud Security Alliance, AI Controls Matrix 1.1.1 (distributed as AICM v1.1), including implementation guidance. https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1

[17] OWASP GenAI Security Project, Top 10 for Agentic Applications 2026. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[18] Center for Internet Security, CIS Benchmarks overview. Select the actual product and version for deployment. https://www.cisecurity.org/cis-benchmarks-overview

[19] NIST, SP 800-207, Zero Trust Architecture. https://csrc.nist.gov/pubs/sp/800/207/final

[20] NIST, SP 800-160 Vol. 1 Rev. 1, Engineering Trustworthy Secure Systems. https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final

[21] NIST, SP 800-55, Measurement Guide for Information Security, Volumes 1 and 2. https://csrc.nist.gov/projects/measurements-for-information-security/publications

[22] The SABSA Institute, SABSA Executive Summary. https://sabsa.org/sabsa-executive-summary/

[23] Model Context Protocol, Specification 2026-07-28, Tools. https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[24] Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020. https://arxiv.org/abs/2005.11401

[25] NVIDIA, OpenShell overview. https://docs.nvidia.com/openshell/about/overview

[26] Microsoft, Microsoft eXecution Container (MXC) documentation. https://github.com/microsoft/mxc

[27] NVIDIA, OpenShell native Windows support documentation. https://github.com/NVIDIA/OpenShell/blob/windows/WINDOWS.md

[28] Cloud Security Alliance, STAR for AI Level 1 Submission Guide, in the AICM v1.1 bundle. https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1

[29] OWASP, Artificial Intelligence Security Verification Standard (AISVS) 1.01. https://github.com/OWASP/AISVS/releases/tag/v1.01

[30] OpenSSH, sshd_config(5), server authentication and session restrictions. https://man.openbsd.org/sshd_config

[31] Apache Software Foundation, Camel Crypto (JCE) Component, digital signature support. https://github.com/apache/camel/blob/main/components/camel-crypto/src/main/docs/crypto-component.adoc

Publication status: AI-CAL v2.0 is a proposed containment assurance framework and conceptual reference architecture. Assess a deployment against its actual boundaries, authorized paths, failure domains, observation, and Qualified Capacity. Publication does not qualify a deployment.